PROJECT REPORT

FloatChat

AI-Powered Conversational Oceanographic Intelligence System

Turning a decade of ARGO float depth, temperature, and salinity profiles into natural-language answers — online or fully offline.

Domain	Oceanographic Data Intelligence & Conversational AI
Core Approach	Hybrid Dual-Engine RAG (Cloud LLM + 100% Offline Local Engine)
Temporal Coverage	2015 – 2025 (10-Year Global History)
Records Indexed	255,000+ oceanographic depth profiles
Core Stack	React · FastAPI · Flask · LangChain · OpenAI GPT · ChromaDB · PostgreSQL/PostGIS

Table of Contents

1. Why This Project?

Oceanographic monitoring is crucial for climate research, monsoon forecasting, marine ecosystem preservation, and maritime navigation. The international **ARGO Program** deploys autonomous robotic floats across global oceans that cycle between the sea surface and 2,000 meters depth, collecting temperature, salinity, and pressure data.

However, accessing and analyzing this valuable data presents major obstacles:

- **Complex binary formats:** data is stored in multi-dimensional NetCDF (.nc) files following Climate and Forecast (CF) metadata conventions.
- **Steep technical barrier:** analyzing the data requires specialized domain tools such as Ocean Data View, MATLAB, or Python xarray/netcdf4.
- **Disconnected search:** non-technical domain researchers, maritime operators, and policy analysts cannot easily ask natural-language questions like “What was the average salinity near Gujarat in winter 2023?”

FloatChat solves this problem by combining Retrieval-Augmented Generation (RAG), Natural Language to SQL synthesis, local NLP semantic parsers, and interactive web visualization into a unified, zero-friction conversational assistant — democratizing access to over 10 years (2015–2025) of depth, temperature, and salinity profiles.

2. System Architecture

FloatChat is built using a modern **three-tier decoupled architecture**:

- **Presentation Layer:** a responsive React UI paired with a Flask proxy server (port 3000).
- **Intelligence / RAG Layer:** a dual-mode FastAPI backend (port 8000) and a Local Fallback Engine (port 5000), built on LangChain, OpenAI GPT, ChromaDB, and local Sentence-Transformers (all-MiniLM-L6-v2).
- **Data Layer:** a PostgreSQL + PostGIS database with spatio-temporal indexes over 255,000+ depth profiles.

Component & Data Flow

```
Client / Presentation Tier
  React Web Dashboard (Port 3000)
  Landing Page & Health Monitor
    | Fetch / Proxy
    v
  Flask Web Server (frontend_server.py)
    | HTTP POST
    v
AI & Orchestration Tier
  Router / Mode Selection
    |-- Cloud Mode (Port 8000) --> FastAPI Server (main.py)
    |                               |-- LangChain SQL Agent (GPT-3.5-Turbo)
    |                               +-- ChromaDB Vector Store (argo_metadata)
    |
    +-- Local / Fallback Mode (Port 5000) --> Flask Backend (backend_server.py)
                                    |-- LocalFloatChatRAG (main_local.py)
                                    |     +-- SmartOceanQuery Regex & Spatial Parser
                                    +-- SentenceTransformer ('all-MiniLM-L6-v2')

Data Storage & Processing Tier
  LangChain Agent  --SQL Execution-->  PostgreSQL + PostGIS (argo_db)
  SmartQuery       --Direct Parameterized SQL-->  PostgreSQL + PostGIS (argo_db)

  Raw NetCDF Files (.nc) --> ETL Pipeline (etl_final.py / load_all_incois_data.py)
                                --Batch Insert--> PostgreSQL + PostGIS (argo_db)
```

Component map: client hooks route every query through a mode selector that chooses between a cloud LangChain/GPT path and a fully offline local RAG path, both of which converge on the same PostGIS-backed data layer.

Architectural Workflow

- **User query submission:** the user submits a natural-language query through the React chat interface.
- **Dual-path intelligence processing — Path A (Online Cloud RAG):** FastAPI leverages LangChain's SQL Database Agent and OpenAI gpt-3.5-turbo to inspect database schemas and generate structured SQL queries, while ChromaDB retrieves relevant domain metadata.
- **Dual-path intelligence processing — Path B (Offline Local RAG / Fallback):** if running locally or without API keys, main_local.py / smart_query.py uses SentenceTransformer embeddings and spatio-temporal regex parsing to map regions (Arabian Sea, Bay of Bengal, Gujarat, etc.), seasons, and measurement types directly into SQL.
- **Execution & synthesis:** the database returns numerical results, and the engine formats the response into natural language with metadata and SQL query transparency.

3. Key Features

Dual RAG Engine (Online & 100% Offline Modes)

- Supports cloud LLMs (OpenAI GPT-3.5) with LangChain tools.
- Includes a self-contained local fallback engine requiring no external APIs or paid credits.

Spatio-Temporal Intelligent Querying

- Understands named geographic regions (e.g., Gujarat, Arabian Sea, Bay of Bengal, Indian Ocean, Chennai, Mumbai).
- Resolves seasonal timeframes (Winter, Monsoon, Summer, Spring, Autumn) and specific calendar years (2015–2025).

Depth-Resolved Oceanic Analytics

- Supports surface layer (0–10m), shallow profiles (0–50m), and deep-sea levels down to 2,060m.

Interactive Glassmorphism Dashboard

- Live query interface, response confidence metrics, live system health monitoring, and sample data cards.

Query & SQL Transparency

- Displays the underlying generated SQL query alongside the conversational answer for scientific auditability.

Automated ETL for Multi-Source ARGO / INCOIS Data

- Extracts, cleans, flattens, and loads multi-dimensional NetCDF profiles in batches with error recovery.

4. Technology Stack

Frontend

Technology	Purpose
React.js (CDN/Standalone)	Dynamic chat components, real-time message stream, and state management
HTML5 & CSS3	Modern dark ocean gradient theme with glassmorphism effects and CSS Grid/Flexbox layouts
Flask (Frontend Server)	Serves templates (landing.html, home.html, chat.html) and acts as an API reverse proxy

Backend & AI Orchestration

Technology	Purpose
FastAPI	Asynchronous, high-performance REST API with OpenAPI/Swagger documentation
LangChain 0.3.27	SQL Database Agent toolkit and hybrid vector retrieval chains
Sentence-Transformers	Local embeddings via all-MiniLM-L6-v2
ChromaDB	Persistent vector database for oceanographic domain metadata and profile indexing
OpenAI API	GPT-3.5-Turbo and text-embedding-3-small (for cloud mode)

Data Engineering & Scientific Processing

Technology	Purpose
xarray & netCDF4	Multi-dimensional scientific array extraction from oceanographic NetCDF files
Pandas & NumPy	Tabular data cleaning, timestamp harmonization, and missing value interpolation
SQLAlchemy	Database connection pooling, ORM mappings, and direct parameterized execution

Database & Storage

Technology	Purpose
PostgreSQL 15+	Relational database storing over 255,000 oceanographic profiles
PostGIS	Geospatial extension supporting geographic point coordinates, bounding boxes, and spatial queries

5. Data Pipeline & Schema

ETL Processing Pipeline (etl_final.py & load_all_incois_data.py)

```
NetCDF (.nc) Files
   |
   v
xarray extraction: Platform ID, Latitude, Longitude, Time, Pressure, Temp, Salinity
   |
   v
Data Cleaning: Timezone normalization, NaN filtering, Quality Control checks
   |
   v
Flattening: Convert 2D arrays (Time x Depth) into 1D relational rows
   |
   v
Batch Insertion via SQLAlchemy Core to PostgreSQL `profiles` Table
```

PostgreSQL Database Schema: profiles

```
CREATE TABLE IF NOT EXISTS profiles (
    id SERIAL PRIMARY KEY,
    filename VARCHAR(255),
    float_id VARCHAR(50),
    profile_index INTEGER,
    depth_index INTEGER,
    timestamp TIMESTAMP,
    latitude DOUBLE PRECISION,
    longitude DOUBLE PRECISION,
    pressure DOUBLE PRECISION,
    temperature DOUBLE PRECISION,
    salinity DOUBLE PRECISION,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Performance Optimization Indexes
CREATE INDEX idx_profiles_float_id ON profiles(float_id);
CREATE INDEX idx_profiles_timestamp ON profiles(timestamp);
CREATE INDEX idx_profiles_location ON profiles(latitude, longitude);
CREATE INDEX idx_profiles_filename ON profiles(filename);
```

Pre-Mapped Geographic Bounding Boxes (smart_query.py)

Region	Latitude Range	Longitude Range
Gujarat Coast	20.0 to 24.5	68.0 to 74.5
Arabian Sea	10.0 to 25.0	60.0 to 75.0
Bay of Bengal	5.0 to 22.0	80.0 to 95.0
Indian Ocean	-40.0 to 25.0	20.0 to 120.0

6. API Endpoints

Frontend Proxy Server (http://localhost:3000)

Method	Endpoint	Description
GET	/	Renders the project landing page
GET	/home	Renders the main dashboard & chat interface
GET	/chat	Renders simplified chat view
POST	/api/query	Proxies chat requests to backend engine
GET	/api/health	Health check for frontend and backend connectivity
GET	/api/dummy-data	Returns sample oceanographic records for UI testing

Local RAG Backend (http://localhost:5000)

Method	Endpoint	Description
POST	/api/rag-answer	Processes queries using local NLP & direct SQL engine (e.g. {"question": "What is the salinity near Gujarat in winter 2023?"})
GET	/api/health	Returns database and embedding model status
GET	/	API metadata and endpoint index

FastAPI Cloud RAG Server (http://localhost:8000)

Method	Endpoint	Description
POST	/query	LangChain SQL Agent + OpenAI query processing (e.g. {"query": "How many ARGO floats are in the database?"})
GET	/health	Backend system status
GET	/docs	Interactive Swagger UI API documentation
GET	/redoc	ReDoc interactive technical documentation

7. Testing Strategy & Verification

Test suites included in the codebase:

Database & Data Verification (test_data.py)

- Verifies PostgreSQL table counts, null value percentages, and temperature/salinity ranges.
- Validates that coordinate values stay within standard geographical bounds.

RAG Pipeline Tests (test_rag.py & enhanced_test_rag.py)

- Tests end-to-end question answering across basic counts, statistical aggregations, and spatial queries.
- Assesses fallback triggers when external LLM endpoints are unreachable.

OpenAI Authentication Validation (test_openai_key.py)

- Confirms validity of OpenAI API credentials and monitors available token quota.

Schema Creation Unit Tests (create_test_table.py)

- Verifies safe creation and teardown of test tables and indexes.

8. Key Design Decisions & Highlights

8.1 Hybrid Dual-Engine Architecture

Rather than creating a brittle system dependent solely on paid cloud LLMs, the project incorporates a deterministic, rule-based SQL generator and a local transformer fallback, so the assistant remains available even without active OpenAI API credits.

8.2 Decoupled Frontend Reverse Proxy

The Flask frontend acts as a proxy for backend microservices, preventing CORS issues and enabling easy switching between backend implementations without changing UI code.

8.3 Direct SQL Generation over Pure Vector Chunking

Numerical time-series and profile data require exact mathematical aggregations (AVG, MIN, MAX). Generating SQL queries over the relational database prevents LLM mathematical hallucinations, while ChromaDB is reserved for semantic metadata search.

8.4 Multi-File Scientific Ingestion Pipeline

Implemented chunk-based processing (BATCH_SIZE = 50) to handle large numbers of NetCDF files without encountering Python memory exhaustion.

8.5 Spatial Bounding-Box Indexing

Pre-defined oceanographic geographical coordinates enable sub-second spatial querying without relying on expensive runtime geocoding.

9. Challenges Faced & Solutions

Challenge	Impact	Engineering Solution
Inconsistent NetCDF Dimensions	Some files used N_PROF/N_LEVELS, others used TIME/PRES/depth.	Built dynamic dimension inspection in etl_final.py that inspects ds.dims and adapts coordinate extraction automatically.
OpenAI Quota & Rate Limits	External LLM API calls could fail during demos or high-traffic periods.	Created main_local.py and smart_query.py with local SentenceTransformer models and regex-driven SQL builders as a zero-cost fallback.
High Memory Usage During Large Ingestion	Ingesting tens of thousands of NetCDF files simultaneously caused out-of-memory errors.	Implemented streaming generator chunking with periodic transaction commits and explicit dataset closing.
Geographic Coordinate Ambiguity	Users ask for “Gujarat”, “Arabian Sea”, or “Chennai” without knowing exact coordinates.	Developed a geographic bounding-box knowledge dictionary in smart_query.py that translates regional names into SQL BETWEEN latitude/longitude clauses.

10. What I Learned

Oceanographic Data Standards

Deep understanding of Climate and Forecast (CF) conventions, NetCDF4 internal hierarchies, ARGO profile structures, and physical units (PSU, dbar, °C).

RAG on Structured Data

Practical experience integrating LLMs with relational SQL engines versus standard unstructured document search.

Resilient AI System Design

Architecting fallback mechanisms (local NLP + rule-based engines) so applications remain functional when third-party APIs are unavailable.

High-Performance Spatial Querying

Utilizing PostGIS and composite B-tree indexes for fast spatio-temporal lookups over hundreds of thousands of rows.

11. Setup & Execution Guide

Prerequisites

- Python 3.10+
- PostgreSQL 14+ with PostGIS
- Git

1. Clone the Repository & Install Dependencies

```
git clone <repo-url>
cd floatchat_prototype
python -m venv venv
.\venv\Scripts\activate   # On Windows
pip install -r requirements.txt
```

2. Configure Environment Variables (.env)

```
DATABASE_URL=postgresql://postgres:<password>@localhost:5678/argo_db
OPENAI_API_KEY=your_openai_api_key_here  # Optional for local mode
```

3. Run ETL Pipeline

```
python etl_final.py
```

4. Launch Application via Unified Launcher (launch.py)

```
python launch.py
# Select Option 3 to start both Backend and Frontend servers
```

5. Access Application

- **Landing Page:** http://localhost:3000
- **Dashboard:** http://localhost:3000/home
- **API Swagger Docs:** http://localhost:8000/docs

12. Future Enhancements

Enhancement	Impact
Live INCOIS satellite telemetry stream	Real-time ingest webhooks to parse new float profiles as they surface and broadcast via satellites
Interactive 3D/2D geospatial trajectory map	Integrate Mapbox/Leaflet to visually plot float trajectories and depth-temperature heatmaps directly inside the dashboard
Biogeochemical (BGC-ARGO) expansion	Ingest and query parameters such as Dissolved Oxygen, pH, Nitrate, and Chlorophyll-A
Autonomous marine anomaly detection	Agentic background jobs that detect ocean temperature anomalies (e.g., Marine Heatwaves) and alert researchers
Multi-agent research assistant	An autonomous oceanographic research assistant coordinating multiple specialized agents

13. Performance Metrics

127.8K–255.6K+
Measurements Ingested

2015–2025
Temporal Coverage (10 Years)

0–2,060 m
Depth Span

>98%
Data Validity

Metric	Value
Total Ingested Measurements	127,825 to 255,650+ records
Temporal Reach	10 years (September 2015 – September 2025)
Float Fleet Coverage	22+ unique ARGO profiling floats across Indian and Southern Oceans
Depth Range	0 (sea surface) to 2,060 meters depth
Data Quality	>98% valid temperature and salinity records
Response Time — Local Engine	50–250 ms (typically < 200 ms)
Response Time — Cloud LangChain LLM Engine	2.0 – 5.0 s

14. Conclusion

FloatChat demonstrates an end-to-end, production-ready solution that transforms raw, complex scientific oceanographic data into actionable conversational intelligence. By uniting modern web engineering, relational and vector databases, and dual-engine RAG pipelines, the system makes deep-sea data universally accessible to scientists, students, and maritime organizations worldwide.

The platform's core strengths can be summarized as:

- Every query is answered through the most reliable available path — cloud LLM when accessible, deterministic local NLP when not.
- Every numerical answer is grounded in exact SQL aggregation rather than LLM approximation.
- Every region name a user might type (Gujarat, Arabian Sea, Chennai...) resolves to precise geospatial bounds.
- Every ingestion run adapts automatically to inconsistent NetCDF file structures.
- Every architectural layer is decoupled, so components can evolve or be replaced independently.

Built with React · FastAPI · Flask · LangChain · OpenAI GPT · ChromaDB · Sentence-Transformers · PostgreSQL & PostGIS · xarray/netCDF4